International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

Agentic Pipeline Operations: LLM Agents for CI/CD Failure Triage and Remediation under Human-in-the-Loop Guardrails

Kartheek Reddy Jinna, Umashanker Bandari

Specialist, LTIMindtree Ltd, Edison, New Jersey, USA

Senior Consultant, Robert Bosch, Chicago, Illinois, USA

ABSTRACT: Failed pipeline runs are a persistent tax on platform engineering teams. In our experience operating continuous integration and delivery (CI/CD) for a large regulated financial-services enterprise, most failures are not novel: they fall into a small set of recurring categories such as dependency resolution errors, flaky tests, expired credentials, image pull and registry faults, exhausted namespace quotas, and drift between declared manifests and live cluster state. Each still interrupts an engineer, who must read logs, correlate recent changes, and apply a well-known fix. This paper proposes an agentic pipeline operations architecture in which a large language model (LLM) agent receives tool-scoped, mostly read-only access to pipeline logs, deployment manifests, run history, and repository history through the Model Context Protocol, a tool interface standard announced in late 2024. Actions are mediated by an autonomy gate that combines a per-action risk score with a calibrated confidence estimate to select one of five levels, from human-only operation to bounded auto-remediation of low-risk classes, and every decision is written to an append-only audit trail. On a replay corpus of 1,240 labelled failures reconstructed in a sandbox Kubernetes and Tekton environment, the agent reaches 87.6% top-1 triage accuracy against 57.1% for a rule-based matcher. Median time-to-green falls from 132 minutes under human-only handling to 30 minutes under bounded autonomy, and calibrated gating holds the auto-remediation false-remediation rate at 0.7%. We present the risk taxonomy, gate design, and practitioner guidance for introducing graded autonomy into regulated delivery platforms.

KEYWORDS: CI/CD, LLM agents, Model Context Protocol, failure triage, auto-remediation, human-in-the-loop, DevOps

I. INTRODUCTION

Continuous integration and continuous delivery have become the operational backbone of software organisations, and the research programme summarised in Accelerate [1] and the annual DORA reports [2] has tied delivery performance to measurable business outcomes. A pipeline that fails and stays red blocks every change queued behind it, so a deployment pipeline is only as useful as the speed with which it returns to green [3]. Empirical studies of CI usage confirm that build breakage is frequent and costly [4], and a comparative study of open-source and financial-organisation build failures found that enterprise pipelines fail for a distinctly different mix of reasons, with a larger share caused by environment, configuration, and integration problems rather than by compilation errors [5].

Our experience supports this. Over several years of running GitHub workflows, Jenkins, and Tekton-based OpenShift Pipelines for hundreds of microservices and containerised legacy middleware in a regulated financial-services setting, we observed that the platform team spent a large and poorly visible fraction of its week on pipeline triage. Most of these failures were recurring. A registry mirror timed out; a HashiCorp Vault lease behind a deploy credential expired; a test with a known timing dependency failed on a busy runner; a namespace hit its ResourceQuota because completed PipelineRun pods and workspaces had not been pruned; a Helm chart drifted from the live object after a manual hotfix. Each has a known remediation, yet each required an engineer to open logs, rebuild context, and act.

Large language models have recently shown that they can interleave reasoning with tool use [6], [7], and can resolve a meaningful fraction of real repository issues when given a carefully designed agent-computer interface [8], [9]. In operations, LLMs have been used to recommend root causes and mitigations for cloud incidents [10] and to drive root cause analysis with retrieved diagnostic context [11]. What is missing for pipeline operations is an engineering pattern that lets such an agent act in a delivery platform that is subject to change control, segregation of duties, and audit,

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

without granting it the broad credentials a human operator holds. The announcement of the Model Context Protocol (MCP) in November 2024 [12] provides a timely building block: a uniform, typed interface through which an agent host connects to narrowly scoped tool servers.

This paper contributes the following:

- An agent architecture for CI/CD failure triage and remediation in which the LLM reaches pipeline logs, manifests, run history, and repository history only through MCP tool servers with explicit read and write scopes, redaction at intake, and an append-only audit trail.
- A risk taxonomy for recurring pipeline failure classes and a scoring function that assigns each (class, action) pair an autonomy ceiling, together with a five-level autonomy model (L0 to L4) adapted from human-automation interaction research.
- A gating rule that combines the risk ceiling with a calibrated confidence score, a pipeline-stage cap, and action budgets, and an analysis of why verbalised model confidence is unsuitable as a gate input.
- An evaluation on a 1,240-failure replay corpus reporting triage accuracy by category, time-to-green by autonomy level, the coverage versus false-remediation trade-off, a tool-context ablation, and a prompt-injection probe, followed by practitioner guidance.

II. BACKGROUND AND RELATED WORK

A. Pipeline failures and their causes

Rausch et al. analysed build logs of Java open-source projects and found that failing tests and compilation dominate, while a notable fraction stems from dependency and environment issues [13]. Vassallo et al. showed that in a financial organisation the distribution shifts towards integration, configuration, and environment-related failures [5], which matches our experience with regulated platforms that route every artefact through controlled registries and secret stores. Humble and Farley framed the deployment pipeline as the single path to production and argued that a red pipeline must be treated as a stop-the-line event [3]. Site reliability engineering adds the discipline of toil reduction: repetitive, automatable manual work should be engineered away [14].

B. Flaky tests and intermittent failures

Flaky tests are a distinct and well-studied failure source. Luo et al. categorised the root causes of flakiness in open-source projects, identifying asynchronous waits, concurrency, and test order dependency as the most common [15]. Parry et al. surveyed the field and noted that rerun-based detection, although costly, remains the dominant industrial approach [16]. Lampel et al. described classifying intermittent CI job failures at scale using job execution features, showing that historical run metadata carries strong signal [17]. We expose run history to our agent as a dedicated tool.

C. LLM agents, tool use, and operational AI

ReAct demonstrated that interleaving reasoning traces with actions improves task success and interpretability [6]. Toolformer showed that models can learn when to call external APIs [7]. SWE-bench established a benchmark of real repository issues [8], and SWE-agent showed that the design of the agent-computer interface strongly affects success rates [9], and Reflexion added verbal self-feedback across attempts [18]. In cloud operations, Ahmed et al. evaluated LLMs for recommending incident root causes and mitigations and reported that engineers found the outputs useful while cautioning about hallucinated specifics [10]; RCACopilot combined handler-driven diagnostic collection with LLM reasoning to predict root-cause categories for production incidents [11]. These systems either advise humans or act in sandboxed benchmarks; our focus is bounded action in a governed delivery platform.

D. Levels of automation and security of LLM-integrated systems

Parasuraman, Sheridan, and Wickens proposed that automation be specified per function (information acquisition, analysis, decision, action) at graded levels [19]. Parasuraman and Riley documented misuse of automation through over-reliance, commonly called automation bias [20]. On the security side, Greshake et al. demonstrated indirect prompt injection, where adversarial text placed in data the model reads can redirect its behaviour [21], and the OWASP LLM Top 10 lists prompt injection and excessive agency among the leading risks [22]. Pipeline logs are exactly such attacker-influenceable input, since any contributor can make a test print arbitrary text. Supply-chain guidance from NIST SP 800-204D [23] and SLSA [24] further requires that CI/CD actions be attributable and that build provenance not be altered by unauthorised actors.

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

III. PROPOSED ARCHITECTURE

Fig. 1 shows the architecture. The central design decision is that the agent never holds a platform credential. It reasons over evidence and emits typed tool calls; MCP servers own the credentials, enforce scopes, and log every call. An autonomy gate outside the model decides whether a proposed action is shown to a human, turned into a pull request, queued for one-click approval, or executed.

A. Intake and redaction

A failure event (a Tekton PipelineRun reaching a Failed condition [25], a GitHub workflow_run event, or a Jenkins notification) is normalised into a record with pipeline identity, stage, commit SHA, target environment, and a log pointer. It then windows the log around the first error marker (by default 400 lines before and 100 after, with a 32k-character cap) and applies redaction. Redaction combines pattern detectors for tokens, private keys, JWTs, and connection strings with an entropy check on long alphanumeric runs, and replaces matches with typed placeholders such as <VAULT_TOKEN_1>.

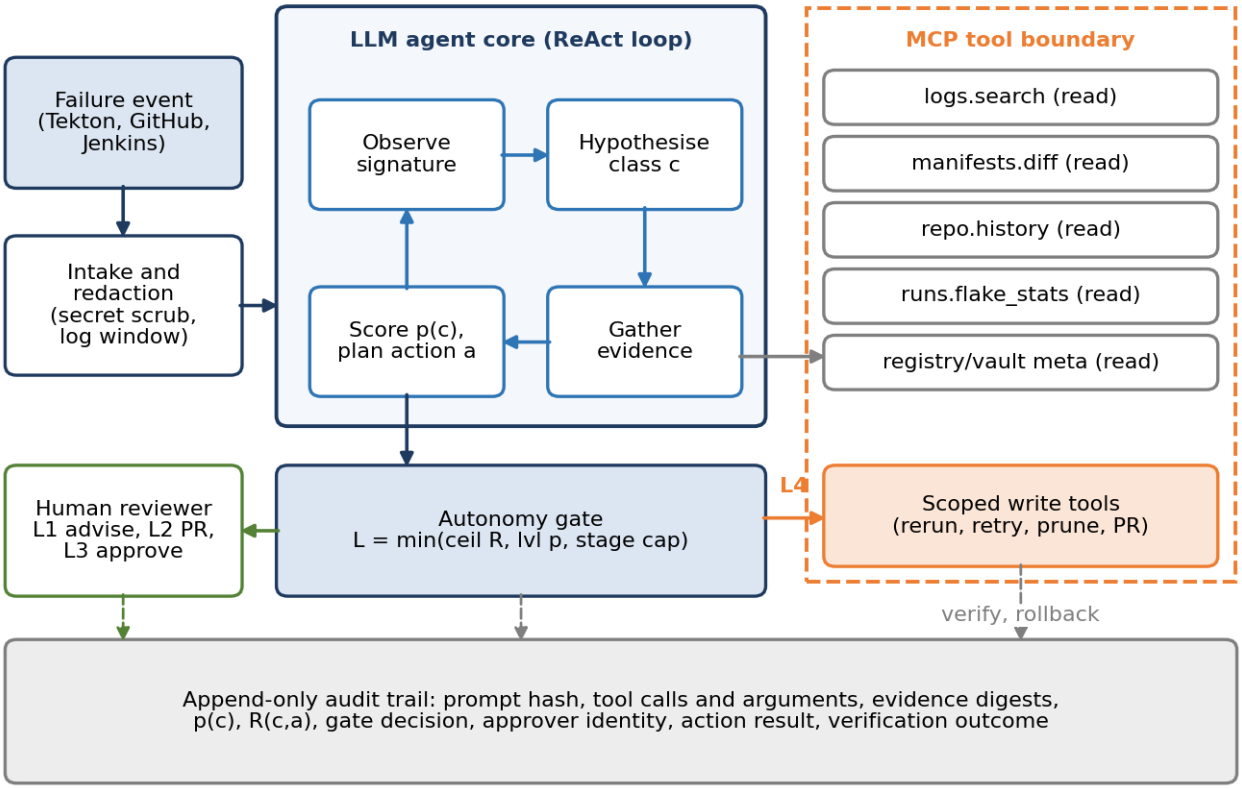


Fig.1. Agentic pipeline operations architecture The LLM agent loop reaches platform systems only through MCP tool servers (dashed boundary); an external autonomy gate routes each proposed action to a human reviewer or to scoped write tools, and all steps are recorded in an append-only audit trail.

B. Agent loop and tool boundary

The agent follows a ReAct-style loop [6] with a hard budget of k = 12 steps and 60k input tokens per incident. At each step it observes the current evidence, updates a hypothesis over the failure classes of Section III-C, and may call a tool to gather further evidence. Tools are exposed through MCP [12], which uses JSON-RPC messages between a host and servers that advertise tools, resources, and prompt templates. We deploy six servers, listed in Table I. Following the lesson from SWE-agent that interface design matters [9], each tool returns compact, pre-summarised output (for example, manifests.diff returns only fields that differ between the rendered manifest and the live object, with managedFields stripped) rather than raw API payloads.

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

Write tools accept a gate token as a mandatory argument. The token is minted by the autonomy gate for one specific (incident, action, arguments) tuple and expires after 10 minutes, so the model cannot invoke a write tool with arguments different from those that were approved. MCP servers authenticate with short-lived, narrowly scoped service identities; the write server may only create PipelineRuns from existing Pipelines, delete completed PipelineRuns, and open pull requests from a bot account whose merges still require CODEOWNERS approval.

Table I. MCP tool servers exposed to the agent and their scopes

Tool (server)	Scope	Returns / effect	Guard
logs.search	Read	Windowed, redacted log lines for a run or step	Run must belong to the incident
runs.flake_stats	Read	Pass-on-rerun rate, last 30 outcomes per test	Aggregates only
repo.history	Read	Commits and diffs since last green run	Diff size cap 400 lines
manifests.diff	Read	Rendered vs. live diff, deprecated APIs, quota use	Namespace allow-list
registry / vault meta	Read	Tag, digest, pull errors; lease TTL (no secret values)	Metadata endpoints only
actions (write)	Write	Rerun, retry via mirror, prune runs, open PR, renew lease	Gate token, budgets, stage cap

C. Risk taxonomy

Autonomy must depend on what an action can break, not only on how sure the model is. We score each candidate action a for a diagnosed class c with a weighted risk function over four factors, each normalised to [0, 1]:

$$R(c, a) = w_1 \cdot B(a) + w_2 \cdot (1 - V(a)) + w_3 \cdot S(a) + w_4 \cdot P(s) \qquad (1)$$

Here B is blast radius (step, pipeline, namespace, shared infrastructure), V is reversibility (rerun 1.0, deletion of completed objects 0.7, code or configuration change 0.3), S is security sensitivity, and P is the proximity of stage s to production. We use $w_1 = 0.35$, $w_2 = 0.25$, $w_3 = 0.25$, and $w_4 = 0.15$, chosen with our change advisory practitioners so that any action touching credentials cannot fall below the L2 threshold. The autonomy ceiling is a step function of R: L4 if $R \leq 0.35$, L3 if $R \leq 0.50$, L2 if $R \leq 0.75$, and L1 otherwise. Table II lists the classes, the permitted actions, and their resulting ceilings for non-production stages.

Table II. Risk taxonomy: failure classes, permitted actions, risk score, and autonomy ceiling (non-production stages)

Class (share)	Typical signals	Permitted action	R	Ceiling
DEP Dependency resolution (21.1%)	Resolve/404 errors, mirror timeout, version conflict	Retry via cache mirror; lock-file pin PR	0.22 / 0.58	L4 / L2
FLK Flaky test (24.0%)	Pass-on-rerun history, timing assertions	Targeted rerun (max 2); quarantine tag + ticket	0.12 / 0.41	L4 / L3
CRD Credential expiry (9.5%)	401/403, x509 expired, Vault lease expired	Request renewal via rotation workflow	0.68	L2
IMG Image pull / registry (13.2%)	ErrImagePull, manifest unknown, rate limit	Retry with backoff or mirror; rebuild tag	0.24 / 0.47	L4 / L3
QTA Resource quota (10.2%)	Exceeded quota, Pending pods, OOMKilled	Prune completed runs; quota change PR	0.44 / 0.66	L3 / L2
DRF Manifest drift (11.5%)	Immutable field, deprecated API, live diff	Reconciliation PR	0.61	L2
OTH Unclassified (10.5%)	No class above threshold	Summary and evidence only	1.00	L1

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

D. Autonomy levels and the gate

Adapting the graded model of Parasuraman et al. [19], we define five levels. L0 is human-only operation (the baseline). At L1 (advise) the agent posts a diagnosis, evidence links, and a suggested fix to the incident channel. At L2 (propose) it additionally prepares the change as a pull request or a ready-to-run command. At L3 (approve-to-act) it executes after a named engineer approves in one click. At L4 (bounded auto) it executes without prior approval, subject to budgets, and notifies after the fact. The gate computes the effective level as:

$$L^*(c, a, s) = \min(\text{Ceil}(R(c, a)), \text{Lvl}(p(c)), \text{Cap}(s), \text{Bud}(a)) \qquad (2)$$

Cap(s) limits autonomy by stage: production-targeting stages are capped at L2 regardless of class, reflecting our change-control obligations, and pre-production stages at L3 for anything that modifies shared state. Bud(a) drops to L2 when an action budget is exhausted: at most two automatic reruns per PipelineRun, five L4 actions per service per day, and a platform-wide circuit breaker that suspends L4 when more than three L4 actions in any 30-minute window fail verification. Lvl(p) maps the calibrated confidence p(c) to a level: L4 requires $p \geq \tau = 0.85$, L3 requires $p \geq 0.70$, and L2 requires $p \geq 0.50$; below that the agent advises only.

E. Calibrated confidence

Verbalised confidence from an LLM is poorly calibrated, as is common for modern neural networks [26]. We therefore compute p(c) from observable signals rather than from the model's stated certainty. The agent samples $m = 5$ independent diagnoses at temperature 0.7 and we measure agreement s with the modal class, following self-consistency [27]. Each class has a short list of evidence predicates (for FLK: the failing test has a pass-on-rerun rate above 5% in 30 days, and no commit since last green touches the test or its code under test), and e is the fraction satisfied by tool outputs; h is the historical precision of the class. A logistic model combines them:

$$p(c) = \sigma(\beta_0 + \beta_1 \cdot s + \beta_2 \cdot e + \beta_3 \cdot h) \qquad (3)$$

The coefficients are fitted with five-fold cross-fitting on the replay corpus so that each incident receives an out-of-fold score.

F. Verification and audit

After any executed action, the verifier waits for the triggered run and checks three conditions: the run reaches a Succeeded condition within a class-specific window, the originally failing step passed rather than being skipped, and the test count did not fall compared with the last green run. If verification fails, the action is marked as a false remediation, reversible effects are rolled back, and the incident is escalated to L1 with the full trace. The audit record stores a hash of the prompt and model version, each tool call with arguments and a digest of its output, p(c), R(c, a), the gate decision, approver identity where applicable, and the verification outcome. It is written to an append-only store and linked from the pipeline run, which gives the attributable action history expected by supply-chain guidance [23], [24].

IV. EVALUATION SETUP

The figures reported in this section and the next are representative values derived from a controlled evaluation environment and practitioner experience; they are illustrative of relative behaviour and are not measurements from any specific production system.

A. Environment

We built a sandbox that mirrors the shape of a regulated enterprise delivery platform: a three-node OpenShift cluster running OpenShift Pipelines (Tekton), self-hosted GitHub Actions runners, and a Jenkins controller, fronted by a JFrog Artifactory instance acting as both remote mirror and internal registry, with HashiCorp Vault issuing dynamic credentials. Thirty-eight services were represented, covering Java microservices, containerised WebSphere Liberty and Tomcat applications, and Python model-serving images. Namespaces carried ResourceQuota objects [28] sized as they would be for team-scoped CI namespaces.

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

B. Replay corpus

We assembled 1,240 failure cases by reconstructing recurring failure patterns observed in practice and re-injecting them in the sandbox. Examples include revoking a mirror route, publishing a transitive version conflict, shortening Vault lease TTLs, deleting image tags, filling quotas with completed PipelineRun pods, and editing live Deployments out of band. Flaky tests were created with realistic asynchronous-wait and order-dependency defects of the kinds catalogued by Luo et al. [15]. The class distribution is shown in Table II. Two engineers labelled every case independently (Cohen's $\kappa = 0.86$), and disagreements were adjudicated. Each case also carries a 'masking' label that marks remediations which would turn the run green while hiding a genuine defect, such as rerunning a deterministically failing test that happens to pass on a different runner.

C. Systems compared

For triage we compare (i) a rule-based matcher with 142 curated regular expressions, representative of the matchers platform teams typically maintain; (ii) the same LLM given only the redacted log window; and (iii) the full agent with all read tools. The agent used a commercially available frontier LLM with native function calling, accessed through an enterprise API gateway with zero data retention. For time-to-green we drew a stratified subset of 240 cases and executed each under five policies, L0 to L4, where the policy fixes the maximum level and the gate may lower it. Human steps were performed by four platform engineers in time-boxed sessions, with acknowledgement delays sampled from previously recorded on-call times for non-paging pipeline alerts. Comparisons are paired.

D. Metrics

Top-1 triage accuracy is the fraction of cases whose predicted class matches the label. Time-to-green (TTG) is the elapsed time from the failure event to the next successful run of the same pipeline at the same commit or its fix commit. The false-remediation rate (FRR) is the fraction of the N_E executed remediations that failed verification (N_F) or were labelled as masking (N_M):

$$FRR = (N_F + N_M) / N_E \qquad (4)$$

Coverage is the fraction of cases with an L4 ceiling that the gate actually executed at L4. Calibration quality is summarised with expected calibration error (ECE) over ten bins [26].

V. RESULTS AND DISCUSSION

A. Triage accuracy

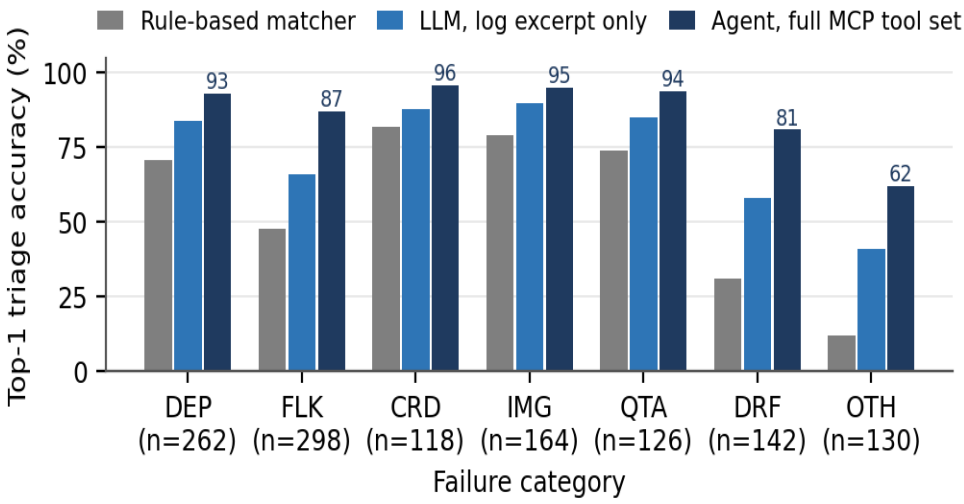


Fig.2. Top-1 triage accuracy by failure category for the rule-based matcher, the LLM with the log excerpt only, and the agent with the full MCP tool set (n = 1,240; values above bars are agent accuracy in percent).

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

Fig. 2 reports top-1 accuracy by category. Overall, the rule-based matcher reaches 57.1%, the log-only LLM 73.5%, and the full agent 87.6%. The gains are not uniform. For classes with distinctive log signatures (CRD, IMG, QTA) rules already perform reasonably and the agent adds 13 to 20 points. The largest gains are for FLK (48% to 87%) and DRF (31% to 81%). In both, the log alone is ambiguous: an assertion timeout looks the same whether the test is flaky or the change broke it, and an immutable-field error does not reveal that a manual edit caused the drift. With runs.flake_stats and repo.history the agent can check whether the test has a history of passing on rerun and whether the change touched it; with manifests.diff it can see the out-of-band field. Unclassified failures remain hard at 62%, which is the intended outcome: these cases have an L1 ceiling, and the value there is a structured evidence summary rather than a label.

B. Time-to-green by autonomy level

Fig. 3 shows TTG distributions for the 240 paired cases and Table III summarises the operational metrics. The median falls from 132 minutes at L0 to 92 at L1, 56 at L2, 40 at L3, and 30 at L4. The L0 to L1 reduction comes from removing context reconstruction, L1 to L2 from removing fix authoring, and L3 from removing the remaining context switch. At L4, 66 of the 240 cases (27.5%) were executed automatically, with a median TTG of about 9 minutes dominated by the rerun duration; the remaining cases followed their L3 path, which is why the L4 box has a long upper whisker and its 90th percentile (86 minutes) is close to that of L3 (89 minutes). Bounded autonomy shortens the common path, not the tail.

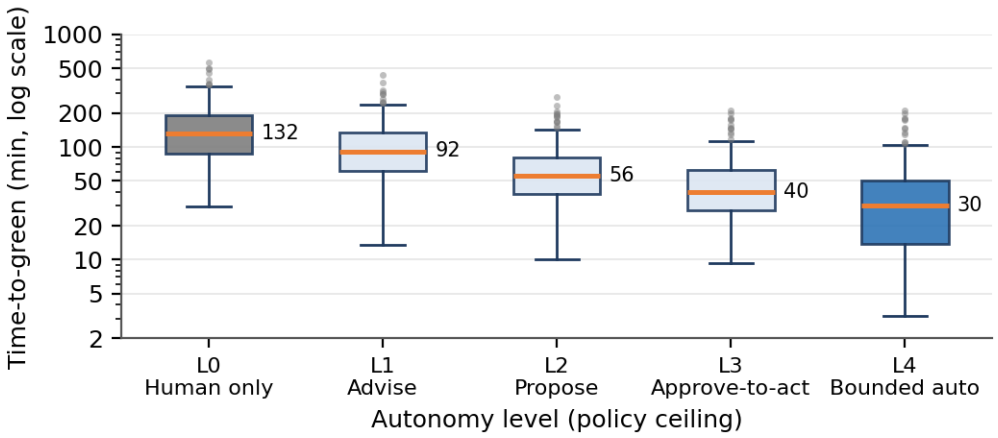


Fig.3. Time-to-green distributions by autonomy policy for 240 paired cases (log scale). Boxes show interquartile range, orange lines the median (labelled in minutes), and whiskers 1.5 × IQR.

Table III. Operational metrics by autonomy policy (240 paired cases; FRR computed over executed remediations in the full corpus replay)

Policy	Median TTG (min)	p90 TTG (min)	Hands-on (min/case)	FRR (%)	Executed without prior approval (%)
L0 Human only	132	259	47	3.3	0
L1 Advise	92	187	31	2.9	0
L2 Propose	56	115	17	2.5	0
L3 Approve-to-act	40	89	8	2.1	0
L4 Bounded auto	30	86	6	1.7	27.5

Two observations in Table III deserve comment. First, humans also produce false remediations: 3.3% of fixes at L0 did not hold or masked a genuine defect, typically a rerun of a failing test that passed on a quieter runner; the agent's flake-history predicates reduce this. Second, FRR at L3 (2.1%) is only modestly below L2 even though every L3 action was

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

approved by an engineer. When we examined the wrongly approved actions, approvers had accepted plausible diagnoses within seconds, a pattern consistent with automation bias [20].

C. Gating threshold and calibration

Fig. 4 shows the trade-off governed by the L4 threshold τ for cases whose ceiling is L4 (about 43% of the corpus). With verbalised confidence as the gate input, the model's stated certainty is high for almost every case, so coverage stays above 80% across most thresholds while FRR remains at 4.2% even at τ = 0.85. With the calibrated score of Eq. (3), coverage at τ = 0.85 is 63.7% and FRR falls to 0.7%. ECE is 0.041 for the calibrated score against 0.187 for verbalised confidence. The calibrated gate thus declines roughly one in three eligible cases, which proceed at L3, in exchange for an auto-remediation path trustworthy enough to leave enabled. We chose τ = 0.85 because below it FRR rises steeply while coverage gains flatten.

D. Value of tool context

Fig. 5 ablates the read tools. Run history and flake statistics add 5.7 points, mostly on FLK; manifest and live-state diffs add a further 4.2 points, mostly on DRF and QTA; and repository history since the last green run adds 4.2 points, mostly on DEP and on separating genuine regressions from flakiness. The mean number of tool calls rises from 2.1 to 5.3 per incident, and mean input context per incident is about 18k tokens with a 95th-percentile end-to-end triage latency of 74 seconds, which is small relative to the minutes saved. As a control we disabled redaction. Accuracy rose by only 0.5 points, which indicates that secret values carry almost no diagnostic information and that redaction costs little.

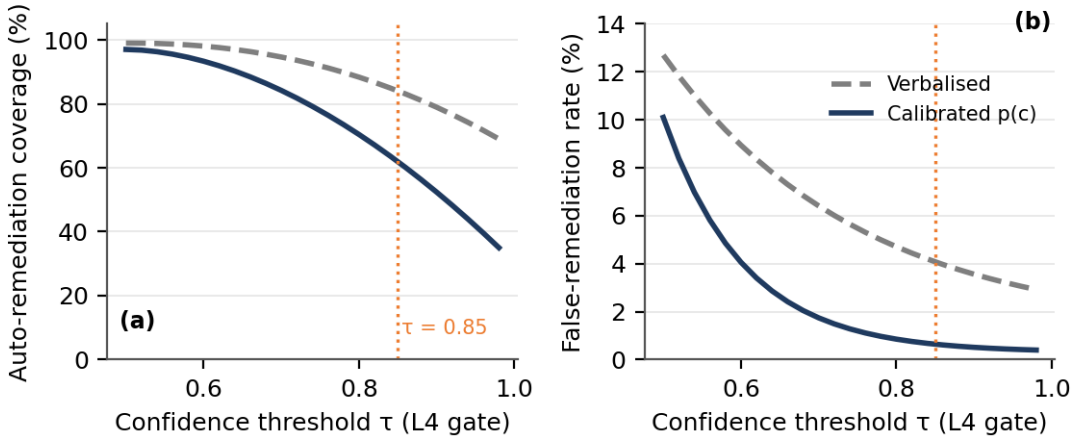


Fig.4. Effect of the L4 confidence threshold τ on (a) auto-remediation coverage of L4-eligible cases and (b) false-remediation rate, for verbalised model confidence and the calibrated score p(c). The dotted line marks the operating point τ = 0.85.

International Journal of Research and Applied Innovations (IJRAI)

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

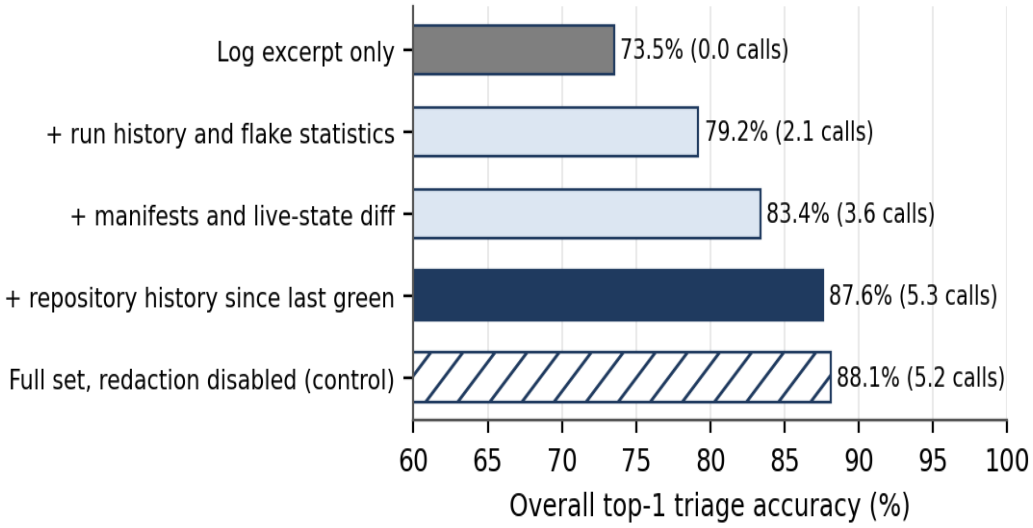


Fig.5. Tool-context ablation: overall top-1 triage accuracy as read tools are added cumulatively, with mean tool calls per incident. The hatched bar is a control run with intake redaction disabled.

E. Prompt-injection probe

Because pipeline logs are attacker-influenceable [21], [22], we added 60 probe cases in which a test printed instructions such as requests to disable a check, print environment variables, or push directly to a protected branch. In 7 of 60 cases (11.7%) the agent's plan incorporated the injected instruction in some form. None was executed: 5 proposed actions had no corresponding write tool, and 2 were rejected by the gate because the arguments fell outside the permitted action set for the diagnosed class. Safety must come from the tool boundary and the gate, not from the model's resistance to manipulation.

F. Discussion

The value of the agent comes less from the model's knowledge of error messages, which the log-only baseline already captures, than from deciding which evidence to fetch. A second finding is that autonomy is best granted per action, not per agent: the same FLK diagnosis justifies an automatic rerun but only an approved quarantine, because quarantining hides signal for every future run.

VI. PRACTITIONER GUIDANCE

We offer the following guidance for teams introducing agents into regulated delivery platforms:

1) **Start at L1 and earn each level per class.** Run advise-only for several weeks and promote a class only against written criteria, for example precision above 95% over at least 50 incidents with no masking remediation.
2) **Keep credentials out of the model's reach.** Put credentials in MCP servers with narrowly scoped identities, and make write tools require a gate-issued token bound to exact arguments.
3) **Make the gate deterministic code.** Risk scores, caps, budgets, and evidence predicates belong in reviewed, version-controlled policy, not in the prompt.
4) **Do not gate on what the model says about itself.** Use self-consistency, evidence checks, and class history, and track calibration error.
5) **Verify outcomes, not actions.** A green run is not proof of a correct fix; check that the failing step ran and the test count did not drop.
6) **Treat approvals as a weak control.** Show falsifying evidence next to the approve button and sample approved actions for review.
7) **Budget and break.** Rerun limits, per-service caps, and a circuit breaker stop an agent from amplifying a systemic outage, such as a registry failure, into hundreds of reruns.

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

VII. LIMITATIONS AND THREATS TO VALIDITY

Construct and external validity. The corpus is a reconstruction of recurring failure patterns in a sandbox, not a sample of production incidents. Failure mixes vary by organisation [5]. Reconstructed failures are likely cleaner than real ones, so absolute accuracy is probably optimistic, although we expect the relative ordering of the three triage approaches to hold.

Human-factor measurement. Human steps were performed in time-boxed sessions by engineers who knew they were being evaluated, and acknowledgement delays were sampled rather than observed.

Model dependence and drift. Results were obtained with one commercial model at one point in time. Model updates can change behaviour without notice; the calibration model of Eq. (3) must be refitted after any model change, and the audit trail records the model version for that reason.

Labelling. The masking label involves judgement, and the risk weights in Eq. (1) reflect one organisation's risk appetite and should be re-derived with local change-control stakeholders.

VIII. CONCLUSION AND FUTURE DIRECTIONS

Recurring pipeline failures are well suited to agentic operation, provided autonomy is granted per action, bounded by deterministic policy, and reached only through narrow tool interfaces. In our evaluation, an LLM agent with MCP-mediated access to logs, run history, manifests, and repository history raised triage accuracy from 57.1% for curated rules to 87.6%, and graded autonomy reduced median time-to-green from 132 to 30 minutes while keeping the false-remediation rate of automatic actions at 0.7% through calibrated gating.

Looking ahead from early 2025, we see four directions. First, as MCP adoption grows, CI engines and registries are likely to ship first-party tool servers, and a shared vocabulary of pipeline tool scopes would make gates portable across organisations. Second, verification could be strengthened by linking remediation records to build provenance so that artefacts built after an automated action carry a signed attestation of that action, extending SLSA-style provenance [24] to operational interventions. Third, an independent critic agent could attempt to falsify each diagnosis before the gate is consulted, countering automation bias in human approval. Finally, the replay corpus methodology could be developed into a shared benchmark for pipeline operations, playing the role for delivery platforms that SWE-bench [8] plays for code repair.

REFERENCES

[1] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. Portland, OR, USA: IT Revolution Press, 2018.
[2] DORA, "Accelerate State of DevOps Report 2024," Google Cloud, 2024.
[3] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.
[4] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2016.
[5] C. Vassallo, G. Schermann, F. Zampetti, D. Romano, P. Leitner, A. Zaidman, M. Di Penta, and S. Panichella, "A tale of CI build failures: An open source and a financial organization perspective," in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2017.
[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023.
[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.
[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in Proc. Int. Conf. Learning Representations (ICLR), 2024.

| ISSN: 2455-1864 | www.ijrai.org | editor@ijrai.org | A Bimonthly, Scholarly and Peer-Reviewed Journal |

||Volume 8, Issue 3, May-June 2025||

DOI:10.15662/IJRAI.2025.0803015

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024.

[10] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in Proc. 45th IEEE/ACM Int. Conf. Software Engineering (ICSE), 2023.

[11] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, "Automatic root cause analysis via large language models for cloud incidents," in Proc. 19th European Conf. Computer Systems (EuroSys), 2024.

[12] Anthropic, "Introducing the Model Context Protocol," Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[13] T. Rausch, W. Hummer, P. Leitner, and S. Schulte, "An empirical analysis of build failures in the continuous integration workflows of Java-based open-source software," in Proc. 14th Int. Conf. Mining Software Repositories (MSR), 2017.

[14] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA, USA: O'Reilly Media, 2016.

[15] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE), 2014, pp. 643-653.

[16] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," ACM Trans. Software Engineering and Methodology, vol. 31, no. 1, Art. 17, 2021.

[17] J. Lampel, S. Just, S. Apel, and A. Zeller, "When life gives you oranges: Detecting and diagnosing intermittent job failures at Mozilla," in Proc. 29th ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE), 2021.

[18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[19] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," IEEE Trans. Systems, Man, and Cybernetics, Part A: Systems and Humans, vol. 30, no. 3, pp. 286-297, 2000.

[20] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," Human Factors, vol. 39, no. 2, pp. 230-253, 1997.

[21] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023.

[22] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications, version 1.1," 2023.

[23] R. Chandramouli, F. Kautz, and S. Torres-Arias, "Strategies for the integration of software supply chain security in DevSecOps CI/CD pipelines," NIST Special Publication 800-204D, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2024.

[24] OpenSSF, "Supply-chain Levels for Software Artifacts (SLSA) specification, version 1.0," 2023. [Online]. Available: https://slsa.dev/spec/v1.0/

[25] The Tekton Authors, "Tekton Pipelines documentation: Tasks, Pipelines, and PipelineRuns," 2024. [Online]. Available: https://tekton.dev/docs/pipelines/

[26] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Machine Learning (ICML), 2017, pp. 1321-1330.

[27] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[28] The Kubernetes Authors, "Resource quotas," Kubernetes Documentation, 2024. [Online]. Available: https://kubernetes.io/docs/concepts/policy/resource-quotas/